Introducción a Python (I)

GIARA

Departamento de Automática y Computación

UPNA

Índice

- Introducción al lenguaje
- Variables
  - Números
    - Enteros
    - Reales
  - Strings
  - Booleanos
- Sintaxis básica
  - IF-ELSE
  - WHILE
  - FOR
- Entrada y salida estándar

Python ¿por qué?

- Principales características de Python
  - Legibilidad
  - Gran cantidad de procedimientos/funciones ‘built-in’
  - Gran cantidad de módulos externos
  - Gran cantidad de estructuras de datos ‘built-in’
  - Multiparadigma
  - Extensible
  - Open source
  - Multiplataforma
  - Gran comunidad de usuarios

IDE

- Spyder 2.3.9
  - IDE para Python con un entorno muy similar al entorno MATLAB
  - Incorpora múltiples paquetes como NumPy, etc.

Compiladores

- Programas cuyo objetivo es realizar la compilación. Este proceso traduce el código fuente de un programa al código máquina [o código intermedio] que directamente entiende un determinado procesador o controlador [o máquina virtual].

- A destacar
- La ejecución es más rápida, más directa y optimizada.
- La compilación detecta errores antes de ejecutar el programa.
- Se pierde la portabilidad entre distintas arquitecturas.
- Aunque técnicamente es diferente se relaciona con el enlazado de dependencias.

Intérpretes y lenguaje interpretado

A diferencia de un compilador (que produce una salida traducida (compilación) un interprete traduce y ejecuta dinámicamente el código fuente. Los lenguajes interpretados son aquellos que soportan este modo de procesado (traducción/ejecución).

A destacar

- La ejecución puede ser más lenta (sobre todo la primera vez).
- Los errores no se detectan hasta alcanzar un determinado punto de la ejecución.
- Facilita la portabilidad entre distintas arquitecturas y sistemas.
- Permite integrar entornos para la extensión y automatización de tareas en otras herramientas.
- Python y Javascript, son lenguajes interpretados.

Estructura

- Estructura básica de un programa
  - **Nivel sintáctico** (*)
    - *Encabezamiento*
      - Inicialización y Comentarios
      - Paquetes y Módulos
    - *Declaraciones*
      - Nombres: variables y constantes
      - Tipos y clases
      - Funciones y procedimientos
    - *Instrucciones*
      - Estructuras de control
      - Llamada a funciones

(*) Con respecto a esta estructura básica, Python es menos riguroso que otros lenguajes

Estructura

```
#/usr/bin/env  python          #Línea de inicialización
"""
  Módulo de ejemplo: este módulo es un ejemplo.
"""
import sys      #Módulos que se importan
import os

debug = True                   #Declaración de variables Globales

class Ejemplo(object):    #Declaración de Clase
    "clase Ejemplo"

def main():          #Declaración de Funciones
    "Main"
    obj= Ejemplo()

if __name__=='__main__':      #Cuerpo Principal
    main()
```

Sintaxis básica

- Primera instrucción en Python
  - print “Hola Python”
- Otra instrucción que probablemente sea de más utilidad
  - help(“print”)
  - help(“algo”)

Variables

- Variables en Python
  - Lenguaje dinámicamente “tipado”
    - Los tipos de las variables se asignan cuando se les da valor a las variables
  - Fuertemente “tipado”
    - Una vez que una variable es de un tipo, no se puede cambiar (tendríamos que redefinir la variable)
    - a=‘3’
    - a+2 -> Produce error
  - Python mantiene en memoria un registro de todas las variables y sus tipos

Variables

- Variable
  - *Nombre o etiqueta que permite acceder al valor de un dato, para leerlo o, en ocasiones, modificarlo*
- En Python todos los nombres o variables pueden ser asociados a nuevos valores.
  - Por convención si el valor de la variable no se va a modificar su nombre se pone en mayúsculas. Permite simular constantes de otros lenguajes.
  - Los nombres “propios” de los valores, sean textuales o numéricos son siempre constantes. Ej.: “Texto”; 12,43
- La variable toma el tipo del valor o valores que contiene.
  - No se declara su tipo, pero puede modificarse expresamente.
- Las variables tienen un ámbito en el que “existen”.

Variables

- Tipo Inmutable
  - Tipos de valores que no permiten cambiar su “estado”.
  - Los tipos simples son, generalmente, inmutables (depende del lenguaje).
  - Al realizar nuevas asignaciones tanto el valor nuevo como el viejo quedan sin cambios (depende del lenguaje).
- Tipo Mutable
  - Tipos de valores que sí permiten cambiar su “estado”.
  - Tienen sentido en datos estructurados, por ejemplo en colecciones, donde su valor es un estado compuesto.
  - Es posible añadir elementos a la colección sin cambiar la asignación de la variable con los datos

Variables

Descripción	Inmutable	Mutable
Entero	Int	…
Real	Float	…
Complejo	Complex	…
Lógico booleano	Bool	…
Errores o excepciones	Exception	…

Variables

- Variables en Python
  - print: sirve para mostrar el valor de la variable
  - type: sirve para saber el tipo de la variable
  - repr: sirve para mostrar la representación del valor mediante un string
    - repr(1.25)

Números

- Python como una gran calculadora
  - 2+2
  - 8-2*2
  - (8-2)*2
  - 8/5 -> Probar
  - 11 % 2
  - 5**2

Números

- Los números enteros
  - Aritmética de números enteros de Python es muy potente y no tiene límite (excepto el de la memoria)
  - En otros lenguajes de programación existe límite (posibilidad de overflow) aunque existan tipos de números grandes (longint, big integer, long long)
  - 2**521 - 1

Números

- Los números reales
  - Normalmente se representa con 64 bits, aunque puede ser modificado por el sistema (sys.float_info).
  - Valores NaN e Inf para “Not a Number” e “Infinito”

- 1.0
- 0.5
- 1.0/10
- 2.0**1023
- 2.0**1024
- 1.0+10**-16
- 1.0+2*10**-16

Números: built-in functions

- Transformación de tipos
  - int y long
- Truncado y redondeo
  - math.trunc, math.round, math.floor, math.ceil
- Otras
  - bin
  - is_integer (para floats)
  - https://docs.python.org/2/library/stdtypes.html#typesnumeric

Strings

- Strings
  - Cadena de caracteres encerrada entre comillas dobles o apóstrofes
    - ‘Hello world’ (apóstrofe)
    - “Hello world” (comillas dobles)
  - Son equivalentes

Strings

- Podemos incluir dobles comillas o apóstrofes dentro del propio string
  - ‘Hello “world” Hello’ (apóstrofe y comillas dobles)
  - “Hello ‘world’ Hello” (comillas dobles y apóstrofe)

Strings

- Concatenación de strings
  - Mediante el operador + (múltiples funcionalidades según el tipo de variable)
  - También se puede realizar simplemente concatenando strings
    - nuevo_str = “Hola” + “qué tal”
    - nuevo_str = “Hola” “qué tal”

Strings

- Caracteres especiales
  - Podemos incorporar caracteres especiales como salto de línea (/n), tabulador (/t)...
- Otra forma de salto de línea
  - Sin necesidad de especificar ningún carácter especial utilizando las triples comillas dobles o el triple apóstrofe
    - ‘’’Hello
      world’’’ (tres apóstrofe)
    - “””Hello
      world””” (tres comillas dobles)

Strings

- Otros caractéres especiales
  - \n (salto de línea)
  - \t (tabulación)
  - \ \ (contrabarra)
  - \’ (apóstrofe)
  - \” (comillas)

Strings

- Acceso a los caracteres de un string
  - s = “Hola”
  - print s[0] -> ‘H’
  - print s[1] -> ‘o’
  - print s[2] -> ‘l’
  - print s[3] -> ‘a’

Strings: built-in functions

- Conversión a string
  - str()
- Repetición *
  - 3*'a' -> 'aaa'
- len
- operador in
  - 'a' in 'casa' -> True
- string.find

Strings: built-in functions

- Otros métodos. Sea x una variable string
  - x.capitalize
  - x.lower
  - x.upper
  - x.count
  - x.find
  - x.startswith
  - x.endswith
  - x.isalnum
  - https://docs.python.org/2/library/stdtypes.html#string-methods

Booleanos

- Booleanos
  - True o False
  - a=False
  - a=True
  - False: equivalente a 0, 0.0, [], diccionario vacío, None
  - True: todo lo demás
- Operadores
  - <, <=, >, >=, ==, !=
  - not, and, or
  - is, is not

Booleanos

- No confundir con operadores sobre bits
  - &
    - 2 & 4 -> 0
  - |
    - 2 | 3 -> 3
  - ^
    - 3 ^2 -> 1

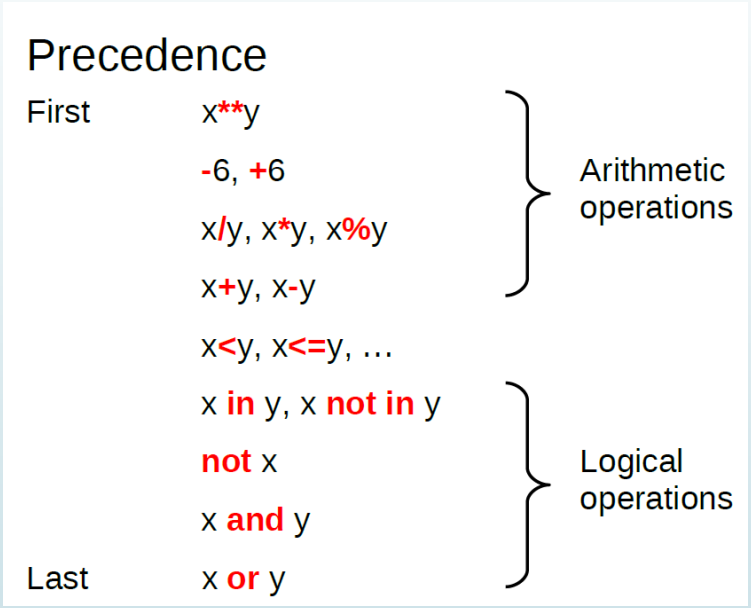
Booleanos

- Booleanos
  - 2 and 4
  - 2 or 4
  - not 2
  - not 4

Booleanos

- Otras expresiones con las que hay que tener cuidado
  - 2-2==1/2
  - True or True and False
  - 7==7/2*2
  - ‘AbC’ > ‘ABC’

Prioridad de operaciones



Prioridad de operaciones

- PEMDAS
  - Parentheses
  - Exponentiation
  - Multiplication and división
  - Addition and Subtraction
- Las operaciones con la misma prioridad se ejecutan de izquierda a derecha (tal como hemos visto antes)
  - 5-3-1 -> 1

Comentarios

- # Esto es un comentario
- x = 5 # Esto es un comentario sobre una línea de código

Built-in functions

- https://docs.python.org/2/library/functions.html

Built-in Functions				
abs()	divmod()	input()	open()	staticmethod()
all()	enumerate()	int()	ord()	str()
any()	eval()	isinstance()	pow()	sum()
basestring()	execfile()	issubclass()	print()	super()
bin()	file()	iter()	property()	tuple()
bool()	filter()	len()	range()	type()
bytearray()	float()	list()	raw_input()	unichr()
callable()	format()	locals()	reduce()	unicode()
chr()	frozenset()	long()	reload()	vars()
classmethod()	getattr()	map()	repr()	xrange()
cmp()	globals()	max()	reversed()	zip()
compile()	hasattr()	memoryview()	round()	__import__()
complex()	hash()	min()	set()	
delattr()	help()	next()	setattr()	
dict()	hex()	object()	slice()	
dir()	id()	oct()	sorted()	

Estructura alternativa: IF-ELSE

```
if condicion:
    instrucción o lista de instrucciones
```

- Aspectos importantes
  - Dos puntos después de la condición
  - Tabulación en la instrucción o listas de instrucciones

Estructura alternativa: IF-ELSE

```
if condición:
    instrucción o lista de instrucciones
else:
    instrucción o lista de instrucciones
```

Estructura alternativa: IF-ELSE

```
if condicion1:
    instrucción o lista de instrucciones
else:
    if condicion2:
        instrucción o lista de instrucciones
    else:
        instrucción o lista de instrucciones
```

Estructura alternativa: IF-ELSE

```
if condicion1:
    instrucción o lista de instrucciones
elif condicion2:
    instrucción o lista de instrucciones
elif condicion3:
    instrucción o lista de instrucciones
else:
    instrucción o lista de instrucciones
```

Estructura repetitiva: WHILE

```
while condición:
    instrucción o lista de instrucciones
```

Estructura repetitiva: WHILE

```
while condición:
    instrucción o lista de instrucciones
else:
    instrucción o lista de instrucciones
```

- Instrucción else final se ejecuta cuando la condición se evalúa a false
  - Si se sale del while con una instrucción break entonces no se ejecuta

Estructura repetitiva: WHILE

```
eps = 1.0
while 1.0 + eps > 1.0:
    eps = eps/2
eps = 2.0*eps
print eps
```

Estructura repetitiva: FOR

```
for variable in elemento_iterable:
    instrucción o lista de instrucciones
```

Estructura repetitiva: FOR

- Elemento iterable
  - Lista explícita

    for i in [0,1,2]:

        instrucción o lista de instrucciones

    for i in [“Hola”, “que”, “tal”]

    for i in “Hola”
  - Función range: muy útil para bucles numéricos
    - range(num): genera num enteros empezando por 0
      - range(5) -> 0,1,2,3,4
    - range([ini], num [,paso]): genera enteros empezando por ini hasta num (sin incluirlo) con una diferencia de paso entre cada número

    for i in range(5)

Estructura repetitiva: FOR

- range(10) -> [0,1,2,3,4,5,6,7,8,9]
- range(5,10) -> [5,6,7,8,9]
- range(0,10,3) -> [0,3,6,9]
- range(-10,-100,-30) -> [-10,-40,-70]

Estructura repetitiva: FOR

- range vs xrange
  - xrange en Python 2.7 es una manera más eficiente de generar un listado de números
  - Funciona exactamente igual pero de manera más "perezosa"
    - Los números se van generando según hacen falta
    - range genera el listado de números desde el principio
  - Útil cuando generamos listas muy grandes o para bucles que van a ser cortados mediante instrucción break

Estructura repetitiva: FOR

- Al igual que while, for también dispone de un bloque else

```
for variable in elemento_iterable:
    instrucción o lista de instrucciones
else:
    instrucción o lista de instrucciones
```

El bloque else se ejecutará únicamente cuando se agote el elemento iterable

Estructura repetitiva: FOR

- Terminación de bucles
  - break: termine el menor bucle que lo envuelve
    - Las instrucciones else no se ejecutarán
  - continue: continúa a la siguiente iteración del bucle
  - pass: no hace nada
    - Únicamente sirve cuando es necesario incluir una instrucción para que el programa sea sintácticamente correcto (while true, ...)

Entrada-salida estándar

- Entrada de datos por teclado
  - variable = raw_input('Mensaje')
    - variable será de tipo string
    - Es necesario transformar a números para manejarlos
      - x = raw_input('Introduce un número: ')
      - Introduce un número: 5
      - x=x+1 -> Error!
  - No confundir con input()
    - Recoge y evalúa lo escrito por el usuario
    - Equivalente a eval(raw_input('Msg'))

Entrada-salida estándar

- Podemos mostrar el resultado de las variables simplemente
  - print ‘El resultado es ‘,x
- Algo más complejo: sustitución de caracteres mediante % (al estilo C)
  - %d, %i: números enteros
  - %e, %E: real con formato exponencial
  - %f, %F: real con formato decimal
  - %c: carácter
  - %r: string obtenido mediante repr()
  - %s: string obtenido mediante str()

Entrada-salida estándar

- x = 5
- print “El resultado de la variable es %d” % x
- print “El resultado de la variable es %d y su cuadrado es %d” %(suma,suma**2)
- print “El resultado de la variable es %f y su cuadrado es %f” %(suma,suma**2)

Entrada-salida estándar

- Una nueva versión que modifica la anterior es mediante str.format
  - s = “Números {} y {}”.format(1,3)
  - s = “Hola {0} y {1}”.format(“Pepe”,”Antonio”)
  - s= “Hola {1} y {0}”.format(“Pepe”,”Antonio”)
  - s = “{!s} {!r}”.format(25.0/3,25.0/3)
  - s = “{!s} {!r}”.format(25.0/3,25.0/3)
    - !s muestra str() y !r muestra repr()
  - s = “{0}{1}{1}{0}”.format(‘a’,’b’)
  - s = “{0}{1}{1}{0}”.format(*‘ab’)
  - https://docs.python.org/2/library/string.html#formatstrings